ELG 5214 Assignment 1: Performance Comparison of JAX and PyTorch

Ethan Xujia Fan

1 Introduction

This report is part of ELG 5214 Assignment 1 and presents a controlled comparison between JAX and PyTorch for training an MLP on the MNIST digit classification task. The objective is to evaluate differences in execution models, compilation strategies, and runtime performance under identical experimental conditions. Both frameworks use the same architecture, optimizer (Adam), learning rate (1e-3), number of epochs (50), dataset split (50k train / 10k validation / 10k test), and batch sizes (64, 256, 1024).

2 Model Architecture

Both JAX and PyTorch implementations use an identical multilayer perceptron (MLP):

$$784 \to 256 \to 128 \to 10$$

ReLU activations are applied after the first two layers, and cross-entropy loss is used. The dataset is split into 50,000 training samples, 10,000 validation samples, and 10,000 test samples. Maintaining identical architecture ensures fairness in comparison.

3 Optimization Strategy

Initially, JAX used stochastic gradient descent (SGD), which resulted in slower convergence and reduced accuracy for large batch sizes. To ensure methodological consistency, JAX was updated to use the Adam optimizer, matching PyTorch. After this modification, both frameworks achieved comparable convergence behavior and final accuracy.

4 Experimental Setup

Experiments were conducted using:

- 50 epochs
- Learning rate = 1e-3
- Batch sizes $\{64, 256, 1024\}$
- Adam optimizer in both frameworks

Metrics measured:

- First epoch time
- Steady-state epoch time
- Final test accuracy

5 Results

5.1 Test Accuracy

Framework	64	256	1024
JAX	0.9819	0.9825	0.9797
PyTorch	0.9812	0.9773	0.9784

5.2 Steady-State Epoch Time (seconds)

Framework	64	256	1024
JAX	0.9175	0.2718	0.1185
PyTorch	1.3670	0.3953	0.1398

6 Analysis

JAX uses JIT compilation via XLA. While the first epoch includes compilation overhead, steady-state execution benefits from graph-level optimization and kernel fusion. PyTorch operates in eager execution mode but remains competitive due to optimized GPU linear algebra backends.

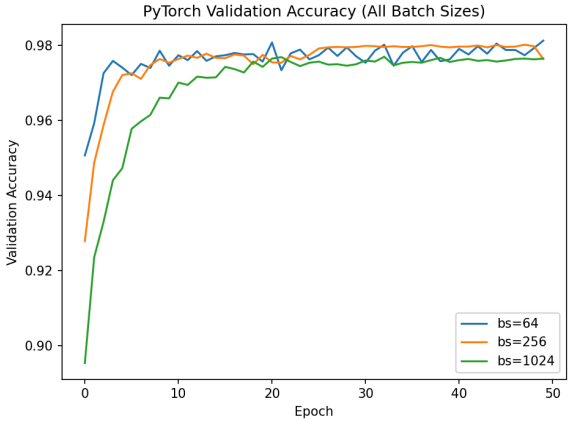
7 AI Usage Reflection

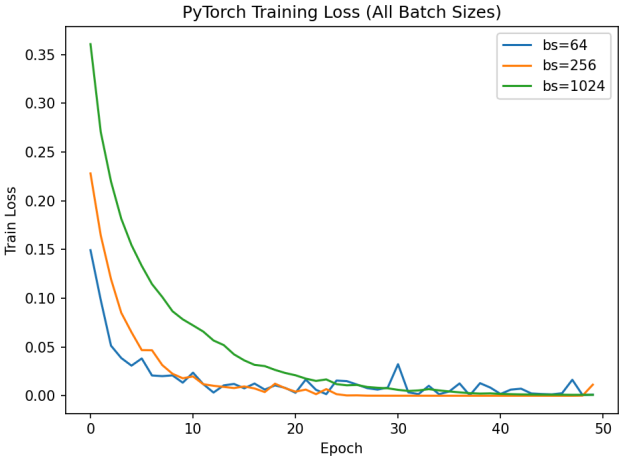
OpenAI ChatGPT was used for debugging, explaining JAX, and tuning.

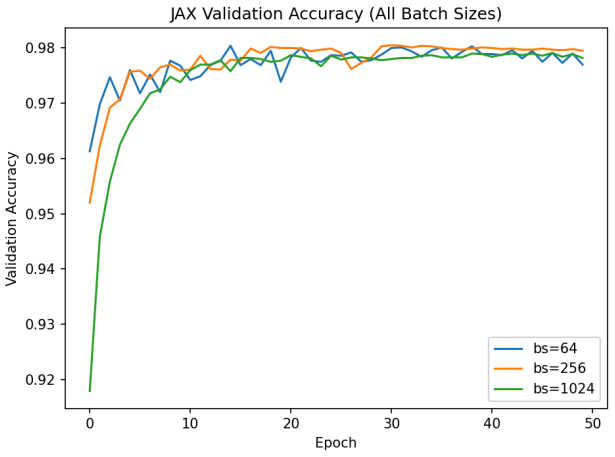
8 Conclusion

With matched architectures and optimizers, both frameworks achieve comparable accuracy. JAX exhibits modest steady-state runtime advantages, while PyTorch remains competitive in both speed and flexibility.

9 Training Curves


PyTorch Validation Accuracy (All Batch Sizes)
Validation Accuracy
Epoch
bs=64
bs=256
bs=1024


PyTorch Training Loss (All Batch Sizes)
Train Loss
Epoch
bs=64
bs=256
bs=1024


JAX Validation Accuracy (All Batch Sizes)
Validation Accuracy
Epoch
bs=64
bs=256
bs=1024


JAX Training Loss (All Batch Sizes)
Train Loss
Epoch
bs=64
bs=256
bs=1024